Advances in Computational Science and Engineering
Vol. 9, 2026, pp. 97-124 (September 3, 2026)
doi:10.3934/acse.2026013

SHAPE OPTIMIZATION USING A NEURAL IMPLICIT GEOMETRY REPRESENTATION

Lukas Freinberger[✉*1], Konstantin Key[✉1], Michael Kofler[✉1], Daniel Breinl[✉2], Welf-Guntram Drossel[✉2], Maria Büttner[✉3], Kristina Roder[✉3] and Stefanie Elgeti[✉1]

[1]Institute of Lightweight Design and Structural Biomechanics, TU Wien, Vienna, Austria

[2]Fraunhofer Institute for Machine Tools and Forming Technology IWU, Chemnitz, Germany

[3]Chemnitz University of Technology,
Professorship of Lightweight Structures and Polymer Technology, Chemnitz, Germany

(Communicated by Alvaro Coutinho)

Abstract. A fundamental aspect of shape optimization is the geometry representation, since it determines the design space explored by the optimizer. Established approaches such as CAD-based parameterizations, free-form deformation, and node-based methods are widely used, but often require a manual, problem-specific setup, restrict the design space, or introduce many variables that require smoothing or filtering. This work presents a neural implicit geometry representation framework for gradient-based shape optimization. Geometries are described by a signed distance function represented by a neural decoder and controlled by a continuous latent field. The decoder is trained once on simple geometric primitives and then kept fixed, so new geometries are reconstructed and optimized by updating only latent control variables. The approach avoids problem-specific retraining and manual definition of design variables, provides local geometric control in a flexible design space, and yields a differentiable path to the extracted surface mesh. Reconstruction experiments show that geometrically diverse shapes can be represented without retraining. In three-dimensional drag-minimization problems, the method produces smooth shape evolutions, outperforms a comparable free-form deformation parameterization, and realizes topological changes. These results indicate that neural implicit geometry representations are a flexible and largely automated alternative to classical shape parameterizations.

1. **Introduction.** Numerical shape optimization is a widely-used approach for improving the performance of engineering designs and has applications in a broad range of fields, including fluid dynamics, structural mechanics, and multi-physics problems [1, 8, 18, 20, 29]. A central aspect of every shape optimization workflow is the parameterization of the geometry. It determines how designs are described, which shape variations are admissible, and how easily the optimizer can explore the design space. Consequently, the choice of geometry parameterization can have a strong influence on the overall shape optimization result.

2020 *Mathematics Subject Classification.* Primary: 49Q10, 65K10; Secondary: 68T07.

Key words and phrases. Shape optimization, gradient-based optimization, neural networks, neural implicit representation, signed distance function.

*Corresponding author: Lukas Freinberger.

A wide range of geometry parameterization approaches has been developed for shape optimization [5, 30]. One common strategy relies on computer-aided design (CAD)-based parameterizations, where the design variables correspond directly to parameters defined in the underlying CAD model [17, 31]. This approach preserves compatibility with industrial design workflows. However, CAD parameterizations are typically tailored to the initial design. The possible geometry variations are often defined manually and may therefore restrict the optimization to changes close to the initial configuration. Furthermore, access to analytical derivatives is often limited within CAD environments, limiting the integration into efficient gradient-based optimization frameworks [31]. Another widely-used parameterization technique is free-form deformation (FFD): Here, the geometry is embedded into a volume spline, whose control points can be repositioned in order to obtain shape changes [15, 23]. FFD provides a smooth parameterization and is well-suited for gradient-based shape optimization. Compared to CAD-based approaches, it also requires less manual construction of problem-specific design variables. However, the admissible shape variations remain strongly linked to the chosen control lattice, which can make the design space restrictive for complex local deformations or large shape changes [31]. An alternative approach are node-based parameterization methods, which use the surface-mesh nodes directly as design variables [2, 9]. These techniques offer significantly larger design spaces. However, the large number of design variables typically requires filtering or smoothing strategies to ensure regular geometries [2].

Overall, while classical geometry parameterization techniques are well-established and have been successfully applied in many engineering problems, their practical use in shape optimization frequently involves substantial manual effort, problem-specific tuning, and limited design spaces. These challenges indicate that geometry representation remains a central bottleneck in automated and scalable shape optimization workflows.

Recent advances in neural networks have opened up new possibilities for geometry representation that are highly promising. In many of these approaches, a neural network is trained to approximate a signed distance function (SDF), whose zero-level set implicitly defines the surface geometry. Park et al. [21] introduced DeepSDF, a neural implicit representation that describes a shape through a compact latent vector. A neural network takes this latent vector together with spatial coordinates and predicts the corresponding signed distance values. This allows memory-efficient representation of shapes, while also enabling interpolation, reconstruction, and shape completion. While DeepSDF provides a global object-level representation, it relies on one latent vector to describe the complete geometry. To improve scalability and local flexibility, Chabra et al. [4] proposed Deep Local Shapes (DeepLS). Instead of using a single global latent vector, DeepLS assigns multiple latent vectors to spatially localized regions (voxels) while sharing the same decoder network. This local decomposition simplifies the learned shape design space and allows detailed reconstruction of complex shapes across a wider variety of geometries.

The use of neural implicit representations in shape optimization has only recently been explored [3, 14, 32]. Lee et al. [16] applied a neural geometry parameterization based on DeepSDF to extrusion die design, demonstrating that learned latent representations can provide greater flexibility than spline-based approaches. However, since a single global latent vector is used, changes in the latent vector may induce non-local changes across the entire domain. Furthermore, the design space

heavily depends on the chosen training data. More recently, Kofler et al. [13] extended neural implicit geometry representations to the structural optimization of lattice structures. Their approach uses a DeepLS-inspired latent representation to describe the geometry of individual unit cells. They demonstrate the applicability of this framework on benchmark examples.

While neural implicit representations have recently been introduced into shape optimization, their practical use remains limited by global latent descriptions, restricted geometry classes, or training data tailored to a specific problem. This motivates a geometry representation that combines the flexibility of neural implicit models with local design control, simple training-data generation, and applicability to different shape classes. Such a representation should also provide a differentiable path from the design variables to the extracted surface, so that it can be integrated into gradient-based optimization.

In this work, we extend the work of Kofler et al. [13] towards a more general shape optimization framework. The central idea is to train a neural decoder on simple geometric primitives and to use a continuous latent field as the geometry design space. During optimization, the decoder remains fixed and only the latent control vectors are updated through gradient-based optimization. Implementing this approach leads to the following main contributions:

- An automated neural implicit geometry representation that avoids problem-specific manual construction of design variables.
- A reusable neural decoder trained on simple geometric primitives, allowing geometrically different shapes to be represented without problem-specific retraining.
- The integration of this representation into a differentiable, gradient-based shape optimization pipeline.

The remainder of this paper is organized as follows. Section 2 introduces the proposed neural implicit geometry representation. It first describes the signed-distance formulation, the continuous latent-field representation, and the training procedure based on geometric primitives. The section then explains how the representation is embedded into a gradient-based shape optimization framework. Section 3 presents the numerical results. First, the reconstruction capability of the pretrained decoder is investigated using a flow-channel geometry and additional geometrically different test cases. The framework is then applied to three-dimensional drag minimization problems, starting from an initial benchmark case, followed by a comparison with a classical FFD parameterization, and finally an example indicating that the implicit representation can also realize topological changes. Section 4 summarizes the main findings, discusses limitations, and outlines directions for future work.

2. **Neural implicit geometry representation and optimization framework.**

This section introduces the neural implicit geometry representation used in this work and explains how it is embedded into a gradient-based shape optimization framework. The geometry is represented by a signed distance function (SDF), whose zero-level set defines the shape boundary. The SDF is approximated by a neural decoder that is controlled by a spatially continuous latent field.

2.1. **Continuous latent-field geometry representation.**

This subsection describes how geometries are represented in the proposed framework. We first introduce the signed distance formulation and the neural decoder, then move from a global latent vector to the continuous local latent field.

2.1.1. *Signed distance functions and neural decoder.* A signed distance function assigns to each point in space the shortest Euclidean distance to the boundary of a domain, together with a sign that indicates whether the point is inside or outside the domain. In this work, negative values are used inside the domain and positive values outside. For a domain $\Omega \subset \mathbb{R}^3$ with boundary $\Gamma = \partial\Omega$, the SDF is written as

$$s(\boldsymbol{x}) = \begin{cases} 0, & \boldsymbol{x} \in \Gamma, \\ -d(\boldsymbol{x}), & \boldsymbol{x} \in \Omega \text{ and } \boldsymbol{x} \notin \Gamma, \\ d(\boldsymbol{x}), & \boldsymbol{x} \notin \Omega \text{ and } \boldsymbol{x} \notin \Gamma. \end{cases} \tag{1}$$

where $d(\boldsymbol{x})$ denotes the Euclidean distance from the spatial point $\boldsymbol{x} \in \mathbb{R}^3$ to the closest point on Γ. Since this distance is zero on the boundary, the shape is described implicitly by the zero-level set

$$\Gamma = \left\{ \boldsymbol{x} \in \mathbb{R}^3 \;\middle|\; s(\boldsymbol{x}) = 0 \right\}. \tag{2}$$

Throughout this work, the boundary Γ is assumed to be piecewise smooth, such that an outward unit normal is defined almost everywhere. The SDF itself remains well defined at edges and corners, although its gradient may not be unique at these locations. Following DeepSDF [21], the signed distance function can be approxi-

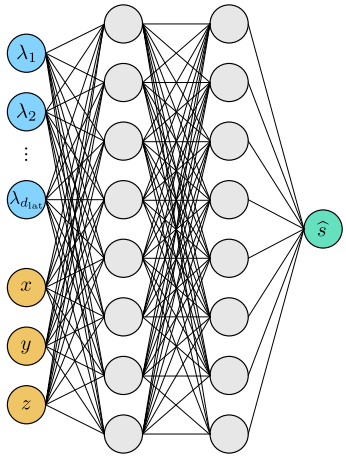


FIGURE 1. Schematic neural decoder for the signed distance function. The first inputs correspond to the components of the latent vector $\boldsymbol{\lambda}$, the last three inputs to the components of the spatial point $\boldsymbol{x}$, and the output to the predicted signed distance value $\widehat{s}(\boldsymbol{x}) = f_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{\lambda})$.

mated by a neural decoder, as illustrated in Figure 1. The decoder takes a spatial point and a latent vector as input and predicts the corresponding signed distance value,

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{\lambda}) \approx s(\boldsymbol{x}). \tag{3}$$

Here, $\boldsymbol{\theta}$ denotes the trainable decoder parameters and $\boldsymbol{\lambda} \in \mathbb{R}^{d_{\text{lat}}}$ is the latent vector of a particular shape. The boundary of a geometry represented by the neural SDF is therefore

$$\Gamma = \left\{ \boldsymbol{x} \in \mathbb{R}^3 \;\middle|\; f_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{\lambda}) = 0 \right\}. \tag{4}$$

By assigning different latent vectors to different shapes while sharing the same decoder parameters, multiple geometries can be represented using the same neural network. In this way, a single global latent vector provides a compact description of the complete shape. However, this also means that changes in the latent vector may affect the entire geometry. In shape optimization, where local design changes are important, this motivates the local representation introduced in the next section.

2.1.2. *Continuous local latent representation.* A first step towards locality is the DeepLS concept of Chabra et al. [4]. Instead of describing the full shape with one global latent vector, the domain is decomposed into local regions, or voxels, and each region is assigned its own latent vector. The boundary can be written as

$$\Gamma = \left\{ \boldsymbol{x} \in \mathbb{R}^3 \;\middle|\; \bigoplus_i w(\boldsymbol{x}, V_i)\, f_{\boldsymbol{\theta}}(T_i(\boldsymbol{x}), \boldsymbol{\lambda}_i) = 0 \right\}. \tag{5}$$

Here, $\boldsymbol{\lambda}_i$ denotes the latent vector associated with the local region V_i, $w(\boldsymbol{x}, V_i)$ weights the contribution of the local region, and $\bigoplus$ combines the local contributions. In [4], the weighting function is chosen as an indicator function, set to 1 if the point $\boldsymbol{x}$ lies inside the corresponding voxel and 0 otherwise. The local transformation maps a global point $\boldsymbol{x}$ into the local coordinate system of voxel V_i by subtracting its center $\boldsymbol{x}_i$:

$$T_i(\boldsymbol{x}) = \boldsymbol{x} - \boldsymbol{x}_i. \tag{6}$$

This local decomposition increases the spatial controllability of the representation while still using a shared decoder. However, because both the latent vector and the local coordinate system are defined piecewise, discontinuities can occur at voxel boundaries. Chabra et al. [4] reduce this effect by including additional samples from neighboring voxels during training. In our numerical experiments, however, such piecewise definitions can still lead to inconsistencies at voxel interfaces. These inconsistencies are problematic for numerical shape optimization, where smooth and differentiable geometry variations are required. We therefore follow the idea of Kofler et al. [13] and replace the piecewise-constant latent codes by a continuous latent field.

The main idea is illustrated in Figure 2. Instead of assigning a constant latent vector to each voxel, the latent representation is defined as a spatially continuous vector-valued field,

$$\boldsymbol{\lambda} : \Omega \to \mathbb{R}^{d_{\text{lat}}}, \qquad \boldsymbol{x} \mapsto \boldsymbol{\lambda}(\boldsymbol{x}). \tag{7}$$

We refer to $\boldsymbol{\lambda}(\boldsymbol{x})$ as the *latent field*. The implicit geometry is then represented as

$$\Gamma = \left\{ \boldsymbol{x} \in \mathbb{R}^3 \;\middle|\; f_{\boldsymbol{\theta}}(\boldsymbol{T}(\boldsymbol{x}), \boldsymbol{\lambda}(\boldsymbol{x})) = 0 \right\}. \tag{8}$$

Here, $\boldsymbol{T}(\boldsymbol{x})$ denotes a continuous coordinate transformation. Its specific form is introduced below. In this work, the latent field is constructed using linear volumetric B-splines, i.e., B-splines of degree $p = 1$, so that the latent field remains continuous across voxel boundaries,

$$\boldsymbol{\lambda}(\boldsymbol{x}) = \sum_{a=1}^{n_{\text{c}}} B_{a,p}(\boldsymbol{x}) \widehat{\boldsymbol{\lambda}}_a, \qquad \widehat{\boldsymbol{\lambda}}_a \in \mathbb{R}^{d_{\text{lat}}}. \tag{9}$$

Here, $B_{a,p}(\boldsymbol{x})$ denotes the B-spline basis function of polynomial degree p associated with the ath latent control point, and $\widehat{\boldsymbol{\lambda}}_a$ is the corresponding latent control vector. This representation is well-suited for optimization for three reasons: First, the linear B-spline interpolation prevents jumps of the latent field across voxel boundaries.

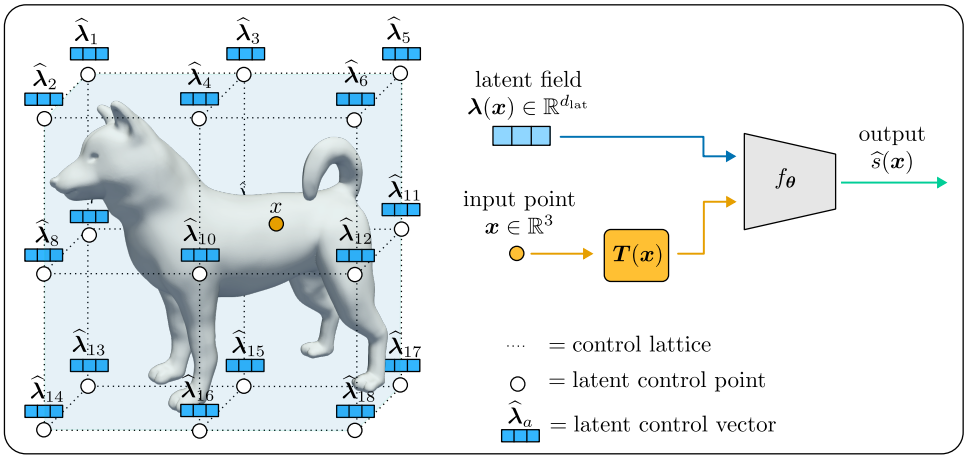


FIGURE 2. Continuous latent-field representation used in this work. Latent control vectors $\widehat{\boldsymbol{\lambda}}_a$ are placed on a control lattice and interpolated to obtain the spatially varying latent field $\boldsymbol{\lambda}(\boldsymbol{x})$. The decoder evaluates the signed distance value from the transformed coordinate $\boldsymbol{T}(\boldsymbol{x})$ and the interpolated latent vector $\boldsymbol{\lambda}(\boldsymbol{x})$. The example geometry shown here has a smooth surface. However, the continuous latent-field representation is not restricted to geometries with smooth boundaries and can also be applied to geometries with edges and corners.

Second, the local support of the B-spline basis preserves localized geometric control. Third, derivatives with respect to both spatial coordinates and latent control vectors follow directly from the B-spline basis functions, which facilitates sensitivity analysis.

Continuity of the latent field alone is not sufficient if the coordinate transformation supplied to the decoder remains discontinuous. Therefore, the local DeepLS transformation in Equation (6) is replaced by the continuous transformation proposed by Kofler et al. [13]. It is defined component-wise for each spatial direction $j \in \{1, 2, 3\}$ as

$$T_j(x_j) = 4 \left| \frac{t_j x_j}{2} - \left\lfloor \frac{t_j x_j + 1}{2} \right\rfloor \right| - 1, \tag{10}$$

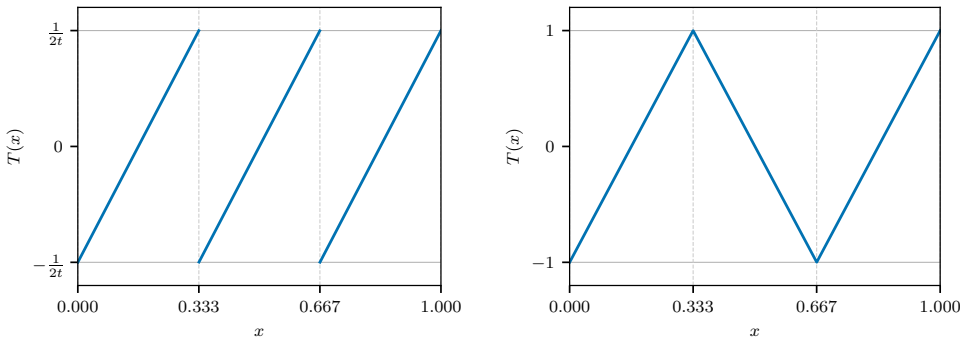
where $\lfloor \cdot \rfloor$ denotes the floor operator, t_j denotes the number of voxels in direction j and x_j is the normalized coordinate in that direction. The full transformation is given by

$$\boldsymbol{T}(\boldsymbol{x}) = (T_1(x_1), T_2(x_2), T_3(x_3))^\top . \tag{11}$$

This hat-function-like transformation maps each voxel to the normalized local interval $[-1, 1]$ while remaining continuous across voxel boundaries. Figure 3 compares both mappings in one spatial direction: Figure 3a corresponds to the piecewise DeepLS transformation in Equation (6), while Figure 3b corresponds to the continuous transformation in Equation (10). Due to the hat-function-like structure, the local coordinate system is mirrored in every second voxel. This preserves locality without the jumps introduced by the piecewise local coordinate system. Since the local representation describes generic local surface patches rather than globally

oriented structures, this alternating reflection does not reduce the representational capability in practice.

The gradient of $\boldsymbol{T}$ is piecewise constant. Therefore, the composed neural SDF is only piecewise differentiable, and the direction of $\nabla\widehat{s}$ can change discontinuously across the interface planes. In a dedicated test, this change was small compared to the surrounding variation, remained localized to the interface planes, and produced no visible artifacts in the reconstructed surfaces. In the same test, a latent field with $p \geq 2$ measurably reduced the effect and slightly improved the reconstruction accuracy. However, at a fixed number of design variables, each variable influences a larger spatial region.



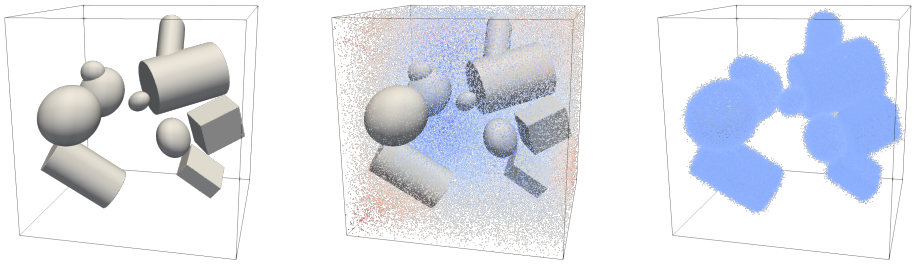
(A) Piecewise local transformation. (B) Continuous transformation.

FIGURE 3. Comparison of local coordinate transformations in one spatial direction. The DeepLS transformation changes discontinuously at voxel boundaries, whereas the hat-function-like transformation remains continuous while still mapping each voxel to a local reference interval.

2.1.3. *Training data generation.* The decoder is trained to learn local shape patches rather than complete global shapes. For this reason, the training data can be generated from simple geometric primitives instead of problem-specific target geometries. One example is shown in Figure 4. Each scene is composed of randomly positioned, oriented, and scaled primitives, such as cubes, spheres, and cylinders, inside the bounding box $\Omega_{\text{box}} = [-1, 1]^3$.

For each scene, SDF sample points are created in two complementary ways. First, points are sampled uniformly in Ω_{box} to cover the volume, as shown in Figure 4b. Second, additional samples are generated close to the geometry surface, as illustrated in Figure 4c, using Gaussian perturbations. This near-surface sampling increases the density of training points around the zero-level set, which is the region most relevant for surface reconstruction. In the present implementation, two perturbation widths are used: a larger one provides samples in a broader near-surface band, while a smaller one concentrates samples very close to the zero-level set.

Using simple geometric primitives for training-data generation has the advantage that the corresponding ground-truth SDF values can be obtained analytically from the implicit descriptions of the individual primitives, which are combined using the minimum operation. For every sampled point $\boldsymbol{x}_k^{(n)}$, this gives the corresponding



(A) Random primitive scene. (B) Uniformly sampled points in the bounding box. (C) Near-surface samples generated by Gaussian perturbations.

FIGURE 4. Training data generation for the neural implicit geometry representation. Each training scene consists of randomly positioned and oriented primitive shapes inside the bounding box $\Omega_{\text{box}} = [-1, 1]^3$. For each scene, points are sampled uniformly in the volume and additionally near the geometry surface using Gaussian perturbations. The signed distance value is evaluated analytically for every sampled point. The explicit surfaces shown here are used only for visualization.

ground-truth signed distance value $s_{\text{gt}}{}_k^{(n)}$. These samples form the dataset

$$\mathcal{D}^{(n)} = \left\{ \left(\boldsymbol{x}_k^{(n)}, s_{\text{gt}}{}_k^{(n)} \right) \right\}_{k=1}^{K}, \qquad n = 1, \ldots, N. \tag{12}$$

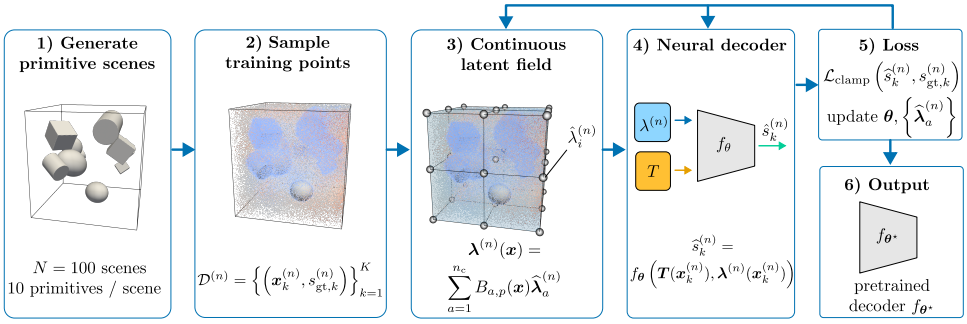
The purpose of this data is not to learn the global shape of a specific object. Instead, the decoder learns local surface behavior from many simple configurations. More complex target geometries can then be represented by combining these local surfaces through the continuous latent field.

2.1.4. *Decoder training and geometry reconstruction.* The overall training and reconstruction procedure is summarized in Figure 5. The upper part of the figure shows the training stage: primitive scenes are generated, SDF samples are drawn, the continuous latent field is evaluated at the sample locations, and the decoder prediction is compared with the analytical SDF value. For each primitive scene, a separate latent field is used, while the decoder parameters are shared. During training, both the decoder parameters and the scene-dependent latent control vectors are optimized.

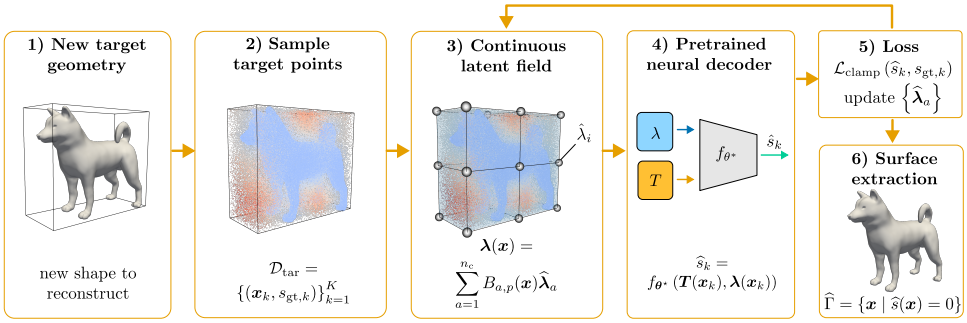
Following DeepSDF [21], we use a clamped L_1 loss,

$$\begin{aligned} \widehat{s}(\boldsymbol{x}) &= f_{\boldsymbol{\theta}}(\boldsymbol{T}(\boldsymbol{x}), \boldsymbol{\lambda}(\boldsymbol{x})), \\ \mathcal{L}(\widehat{s}(\boldsymbol{x}), s_{\text{gt}}(\boldsymbol{x})) &= |\text{clamp}(\widehat{s}(\boldsymbol{x}), \delta) - \text{clamp}(s_{\text{gt}}(\boldsymbol{x}), \delta)|. \end{aligned} \tag{13}$$

Here, $\delta > 0$ is the truncation distance and $\text{clamp}(a, \delta) = \min(\delta, \max(-\delta, a))$. The clamp limits the influence of large distance values far away from the surface. This is useful because the exact SDF values far from the zero-level set are less important for the extracted geometry than the values close to the surface.



(A) Training of the neural decoder using geometric primitives.



(B) Reconstruction of latent control vectors for a new target geometry using the pretrained decoder.

FIGURE 5. Training and reconstruction workflow. During training, the decoder parameters and scene-dependent latent control vectors are optimized. During reconstruction, the decoder is fixed and only the latent control vectors are optimized for the target geometry.

Using the datasets from Equation (12), the training problem is written as

$$\left(\boldsymbol{\theta}^\star, \left\{\hat{\boldsymbol{\lambda}}_a^{(n),\star}\right\}_{n,a}\right) = \underset{\boldsymbol{\theta}, \left\{\hat{\boldsymbol{\lambda}}_a^{(n)}\right\}}{\arg\min} \sum_{n=1}^{N} \left[\sum_{k=1}^{K} \mathcal{L}\left(\hat{s}_k^{(n)}, s_{\mathrm{gt}}{}_k^{(n)}\right) + \beta \sum_{a=1}^{n_c} \left\|\hat{\boldsymbol{\lambda}}_a^{(n)}\right\|_2^2\right], \tag{14}$$

$$\hat{s}_k^{(n)} = f_{\boldsymbol{\theta}}\left(\boldsymbol{T}(\boldsymbol{x}_k^{(n)}), \boldsymbol{\lambda}^{(n)}(\boldsymbol{x}_k^{(n)})\right).$$

The regularization term penalizes large latent coefficients and is weighted by β. This follows the same idea as in DeepSDF, where the latent codes are regularized to avoid unnecessarily large latent values.

After training, the decoder parameters are fixed at $\boldsymbol{\theta}^\star$. The lower part of Figure 5 shows the reconstruction stage for a new target geometry. In this step, the decoder is not retrained. Instead, target SDF samples are generated for the new geometry and only the latent control vectors are optimized such that the neural SDF matches these samples.

Before the target SDF samples are generated, the bounding box of the target geometry is scaled uniformly by $2/L$, where L denotes its largest extent. The SDF values are computed in this isotropically scaled coordinate system. This preserves the aspect ratio and maps the largest extent to 2, consistent with the convention used for the training scenes in $\Omega_{\mathrm{box}} = [-1, 1]^3$. Consequently, the resulting SDF

values have a characteristic scale comparable to that of the training data, and the clamping distance $\delta = 0.1$ retains a consistent relative meaning.

The reconstruction problem is

$$\begin{aligned} \left\{\widehat{\boldsymbol{\lambda}}_a^\star\right\}_{a=1}^{n_\mathrm{c}} &= \underset{\{\widehat{\boldsymbol{\lambda}}_a\}_{a=1}^{n_\mathrm{c}}}{\arg\min} \left[\sum_{k=1}^{K} \mathcal{L}\big(\widehat{s}_k, s_{\mathrm{gt}\,k}\big) + \beta \sum_{a=1}^{n_\mathrm{c}} \left\|\widehat{\boldsymbol{\lambda}}_a\right\|_2^2\right], \\ \widehat{s}_k &= f_{\boldsymbol{\theta}}(\boldsymbol{T}(\boldsymbol{x}_k), \boldsymbol{\lambda}(\boldsymbol{x}_k)), \qquad \boldsymbol{\theta} = \boldsymbol{\theta}^\star \text{ fixed}. \end{aligned} \tag{15}$$

The result of this step is a set of latent control vectors that reconstructs the initial geometry in the latent design space. These latent control vectors are then used as the starting point for the shape optimization. For visualization and for the later numerical analysis, the zero-level set of the reconstructed neural SDF is converted into an explicit surface mesh using the differentiable dual marching cubes method FlexiCubes [24].

2.1.5. *Decoder architecture.* For the decoder $f_{\boldsymbol{\theta}}$, we use a fully connected feed-forward architecture based on the one proposed in DeepSDF [21]. The input is the concatenation of the transformed coordinate $\boldsymbol{T}(\boldsymbol{x})$ and the interpolated latent vector $\boldsymbol{\lambda}(\boldsymbol{x})$, and the output is a single scalar signed distance value. ReLU activations are used between the hidden layers, while the output layer is kept linear so that the predicted distance is not restricted to a bounded range. The decoder weights $\boldsymbol{\theta}$ and the scene-dependent latent control vectors are trained jointly using the Adam [11] optimizer. The concrete architecture and training parameters used in the numerical experiments are reported in Section 3.1.1.

2.2. **Optimization framework.** The neural implicit geometry representation introduced in Section 2.1 is embedded into a gradient-based shape optimization framework. In the following, we first formulate the general optimization problem and then describe how the proposed neural parameterization is integrated into the optimization loop.

In partial differential equation (PDE)-constrained shape optimization, the geometry is described by a vector of design variables $\boldsymbol{d}$, which implicitly define the computational domain $\Omega(\boldsymbol{d})$. The objective is to determine the optimal design parameters such that a given function is minimized while satisfying the governing equations and additional constraints. In the present framework, the vector of design variables is obtained by concatenating all latent control vectors which define the continuous latent field introduced in Equation (9). The optimization problem can therefore be written as

$$\begin{aligned} \min_{\boldsymbol{d}} \quad & J(\boldsymbol{q}(\boldsymbol{d}), \Omega(\boldsymbol{d})) \\ \text{s.t.} \quad & \mathcal{R}(\boldsymbol{q}(\boldsymbol{d}), \Omega(\boldsymbol{d})) = \boldsymbol{0}, \\ & \mathcal{C}_i(\boldsymbol{q}(\boldsymbol{d}), \Omega(\boldsymbol{d})) \le 0, \quad i = 1, \dots, n_\mathrm{g}. \end{aligned} \tag{16}$$

Here, $\boldsymbol{q}$ denotes the state vector obtained from the solution of the governing PDEs expressed in residual form by $\mathcal{R}$. For example, in the case of the incompressible flow problem considered later, this state consists of the velocity and pressure fields, i.e., $\boldsymbol{q} = (\boldsymbol{u}, p)$. The functions $\mathcal{C}_i$ represent additional constraints.

The resulting optimization workflow is shown in Figure 6. It is divided into an initialization stage and the iterative online optimization loop.

Initialization stage: Geometry reconstruction. Before the actual shape optimization, an initial geometry needs to be provided. In particular, one needs to provide an

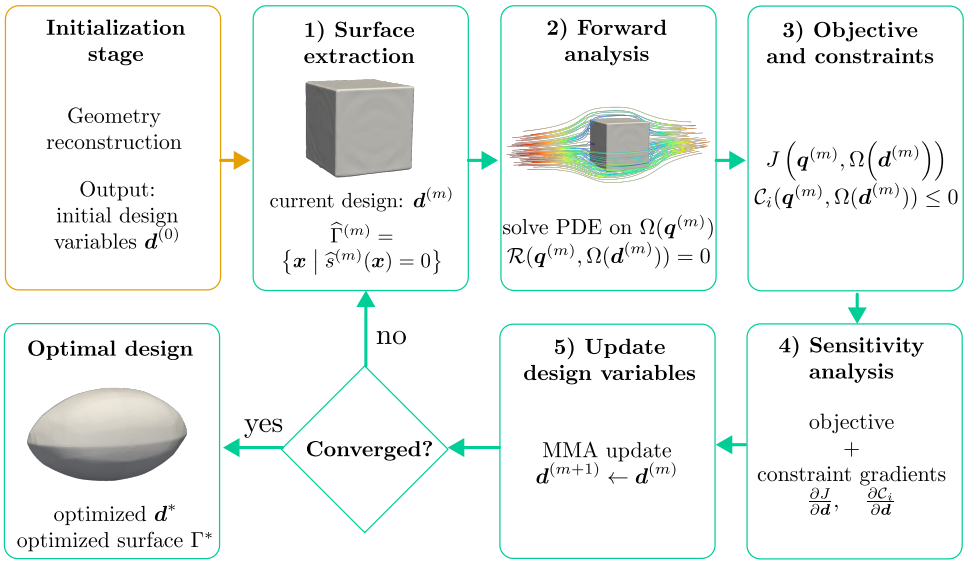


FIGURE 6. Schematic illustration of the proposed shape-optimization workflow. The initialization stage reconstructs the initial geometry and provides the initial latent design variables. The online stage iteratively extracts the current surface, solves the governing PDE, evaluates objective and constraints, computes sensitivities, and updates the design variables until convergence.

initial geometry described via latent codes. Since it is not obvious to a domain expert, which combination of latent values would provide a good initial geometry as the latent design space is non-intuitive, the initial geometry is provided in a more human-understandable format like a CAD representation. For this geometry, one then has to determine the latent code representation using the following procedure: With the decoder fixed at $\boldsymbol{\theta}^\star$, the reconstruction problem in Equation (15) is solved for the initial design. Only the latent control vectors are optimized in this step. The resulting latent control vectors provide the initial design variables $\boldsymbol{d}^{(0)}$.

For shape optimization, the prescribed design domain is used as the bounding box for the isotropic scaling described in Section 2.1.4, and this scaling remains fixed throughout the optimization.

Step 1. Surface extraction. At optimization iteration m, the current design vector $\boldsymbol{d}^{(m)}$ defines the latent control vectors and therefore the latent field $\boldsymbol{\lambda}^{(m)}(\boldsymbol{x})$. Together with the fixed decoder, this gives the current neural signed distance function

$$\widehat{s}^{(m)}(\boldsymbol{x}) = f_{\boldsymbol{\theta}}\left(\boldsymbol{T}(\boldsymbol{x}), \boldsymbol{\lambda}^{(m)}(\boldsymbol{x})\right). \tag{17}$$

The current surface is obtained as the zero-level set of this field and is converted into an explicit surface mesh using the differentiable FlexiCubes algorithm [24]. This provides the reconstructed boundary $\widehat{\Gamma}^{(m)}$ and the computational domain $\Omega^{(m)}$ used for the subsequent analysis.

Step 2. Forward analysis. The governing PDEs are solved on the current domain $\Omega^{(m)}$. In this step, one numerically determines the state $\boldsymbol{q}^{(m)}$ that reduces the PDE

to zero:

$$\mathcal{R}\left(\boldsymbol{q}^{(m)}, \Omega^{(m)}\right) = \mathbf{0}. \tag{18}$$

Depending on the application, this analysis can be performed for example by a finite element or finite volume discretization.

Step 3. Objective and constraint evaluation. After the state has been computed, the objective functional and the constraint functions are evaluated on the current design,

$$J^{(m)} = J\left(\boldsymbol{q}^{(m)}, \Omega^{(m)}\right), \qquad \mathcal{C}_i^{(m)} = \mathcal{C}_i\left(\boldsymbol{q}^{(m)}, \Omega^{(m)}\right), \quad i = 1, \dots, n_{\mathrm{g}}. \tag{19}$$

These values are passed to the optimizer and are also used to check the convergence and feasibility of the current design.

Step 4. Sensitivity analysis. The sensitivity analysis follows the differentiable path from the latent design variables to the objective and constraint values,

$$\boldsymbol{d} \to \boldsymbol{\lambda}(\boldsymbol{x}) \to s(\boldsymbol{x}) \to \widehat{\Gamma} \to \{J, \mathcal{C}_i\}, \tag{20}$$

which enables gradient-based optimization. The total derivative with respect to the latent design variables then follows schematically from the chain rule

$$\frac{\mathrm{d}J}{\mathrm{d}\boldsymbol{d}} = \frac{\partial J}{\partial \widehat{\Gamma}} \frac{\partial \widehat{\Gamma}}{\partial s} \frac{\partial s}{\partial \boldsymbol{\lambda}} \frac{\partial \boldsymbol{\lambda}}{\partial \boldsymbol{d}}. \tag{21}$$

The first factor, $\frac{\partial J}{\partial \widehat{\Gamma}}$, denotes the surface shape sensitivity of the CFD objective. It is obtained from a continuous-adjoint analysis. The remaining factors describe the differentiable geometry representation. The factor $\frac{\partial \widehat{\Gamma}}{\partial s}$ represents the sensitivity of the extracted surface with respect to the underlying neural SDF. In the present work, this information is provided by FlexiCubes [24], which allows gradients to be propagated through the surface extraction step by automatic differentiation. The derivative $\frac{\partial s}{\partial \boldsymbol{\lambda}}$ is also obtained by automatic differentiation of the neural decoder with respect to its latent input. Finally, $\frac{\partial \boldsymbol{\lambda}}{\partial \boldsymbol{d}}$ follows directly from the B-spline interpolation in Equation (9), because the design variables are the latent control vectors. The same differentiation path is used for the constraint gradients $\mathrm{d}\mathcal{C}_i/\mathrm{d}\boldsymbol{d}$, with the first factor replaced by the corresponding surface sensitivity of each constraint.

Step 5. Update of design variables. The current objective value, constraint values, and their gradients are supplied to the optimizer. In this work, the Method of Moving Asymptotes (MMA) [25] is used for the design-variable update. MMA is a well-established gradient-based optimization algorithm for constrained nonlinear optimization problems. At each iteration, it constructs a local convex approximation of the original optimization problem using the current function values and sensitivity information.

The optimizer computes an update of the latent design variables and returns a new design vector $\boldsymbol{d}^{(m+1)}$. If the convergence criterion is not satisfied, the workflow returns to the surface-extraction step and repeats the online loop. Once the convergence criterion is met, the final latent design vector defines the optimized neural SDF and the corresponding optimized geometry.

3. Numerical results.

This section evaluates the proposed neural implicit geometry representation in two steps. First, its reconstruction capability is evaluated for a number of test cases of broad geometrical variety. The framework is then applied to three-dimensional drag-minimization problems to study its performance in

gradient-based shape optimization. The performance study includes a comparison with free-form deformation and an example involving topological changes.

3.1. **Geometry reconstruction.** Before applying the proposed neural implicit representation in the shape optimization framework, we first evaluate its reconstruction capability. This is an important intermediate step, since the optimization procedure relies on an accurate reconstruction of the initial geometry in the latent design space. Additionally, the reconstruction experiments are used to assess the geometric flexibility of the proposed representation approach. In this section, we therefore investigate three aspects. First, we analyze the training behavior of neural decoders with different latent dimensions. Second, we study the reconstruction accuracy based on a fluid-flow channel geometry, and finally, we demonstrate that the same pretrained decoder can reconstruct geometries that differ significantly from each other.

3.1.1. *Network training.* All networks are trained using the data-generation procedure described in Section 2.1.3. The specific data-generation parameters and the network and training parameters used for the numerical experiments are summarized in Tables 1 and 2. For each training scene, a continuous latent field is defined on a latent control lattice with $9\times9\times9$ latent control points, corresponding to $8\times8\times8$ voxels. We train three decoder networks with latent dimensions $d_{\text{lat}} \in \{8, 16, 32\}$.

TABLE 1. Data-generation setup used for the numerical experiments.

Parameter	Value
Number of training scenes	$N = 100$
Primitives per scene	10
Primitive types	Cubes, spheres, cylinders
Uniform samples per scene	100,000
Near-surface samples per scene	1,000,000
Gaussian standard deviations	$\sigma_1 = 0.025$, $\sigma_2 = 0.001$

TABLE 2. Decoder and training setup used for the numerical experiments.

Parameter	Value
Latent dimensions compared	$d_{\text{lat}} \in \{8, 16, 32\}$
Hidden layers	4
Units per hidden layer	128
Activation function	ReLU
Output layer	Linear
Dropout	None
Training epochs	10 000
Learning rate	10^{-3}
Clamping distance	$\delta = 0.1$
Code-regularization weight	$\beta = 10^{-4}$

In this work, one epoch denotes one pass over the $N = 100$ training scenes. For each scene, one optimizer step is performed using a freshly drawn random subset of 4096 of its 1.1 million samples; hence, an epoch does not constitute a full pass over all samples. Consequently, the 10 000 training epochs correspond to 10^6 optimizer steps. Training took 4 h 25 min, 4 h 26 min, and 4 h 27 min for $d_{\mathrm{lat}} = 8$, 16, and 32, respectively, on a single NVIDIA GeForce RTX 4090 with 24 GB of memory.

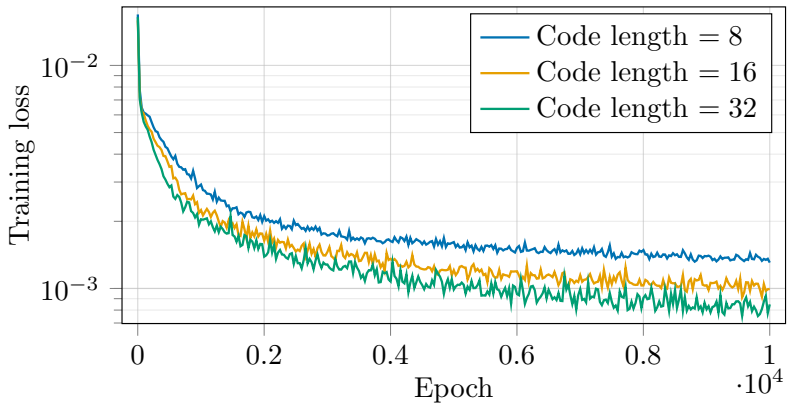


FIGURE 7. Training loss over 10 000 epochs for decoder networks with latent dimensions $d_{\mathrm{lat}} = 8$, 16, and 32.

The training histories of the three networks are shown in Figure 7. All networks show a rapid decrease of the loss during the initial training phase, followed by a slower convergence towards a nearly constant final value. Increasing the latent dimension improves the final training loss, indicating that larger latent control vectors provide a more expressive local representation. The difference between the networks is most pronounced between $d_{\mathrm{lat}} = 8$ and $d_{\mathrm{lat}} = 16$, whereas the additional improvement from $d_{\mathrm{lat}} = 16$ to $d_{\mathrm{lat}} = 32$ is smaller. This suggests that the representation capacity starts to saturate for the considered training data, although the network with $d_{\mathrm{lat}} = 32$ still achieves the lowest training loss.

A classical test-set loss is not used here, because the latent control vectors are optimized jointly with the decoder for each training scene. For a new geometry, the relevant question is therefore not only how well the decoder predicts unseen SDF samples, but whether the fixed pretrained decoder can reconstruct an unseen target geometry by optimizing only its latent control vectors. For this reason, generalization is evaluated in the following reconstruction experiments, where the decoder is kept fixed and only the latent control vectors are optimized for target geometries that were not part of the training data.

3.1.2. *Reconstruction accuracy measures.* To quantify reconstruction accuracy, we use two separate error measures based on the mean absolute error (MAE), capturing both the accuracy of the neural signed distance field and the geometric accuracy of the extracted explicit surface. The first metric evaluates the signed distance field directly. Given the ground-truth SDF s_{gt} and the reconstructed SDF $\widehat{s}(\boldsymbol{x}) =$

$f_{\boldsymbol{\theta}}(\boldsymbol{T}(\boldsymbol{x}), \boldsymbol{\lambda}(\boldsymbol{x}))$, the field error is defined as

$$\begin{aligned} \mathcal{X}_\tau &= \{\boldsymbol{x} \in \Omega \ : \ |s_{\mathrm{gt}}(\boldsymbol{x})| \leq \tau\}, \\ \mathrm{MAE}_{\mathrm{SDF}} &= \frac{1}{|\mathcal{X}_\tau|} \sum_{\boldsymbol{x}_k \in \mathcal{X}_\tau} |\widehat{s}(\boldsymbol{x}_k) - s_{\mathrm{gt}}(\boldsymbol{x}_k)|, \end{aligned} \tag{22}$$

where the evaluation is restricted to a narrow band around the surface, with $\tau = \delta = 0.1$, consistent with the chosen truncation distance δ used in the training loss. $|\mathcal{X}_\tau|$ denotes the number of sample points contained in the narrow band. This restriction focuses the error measure on the region relevant for surface reconstruction and avoids domination by samples far away from the interface.

As a second metric, we evaluate the geometric error of the extracted surface. With $\widehat{\Gamma}$ denoted as the reconstructed zero-level set after surface extraction and with $\mathcal{V}(\widehat{\Gamma})$ being the corresponding set of mesh vertices, the geometric error is defined as

$$\mathrm{MAE}_{\mathrm{geom}} = \frac{1}{|\mathcal{V}(\widehat{\Gamma})|} \sum_{\boldsymbol{v} \in \mathcal{V}(\widehat{\Gamma})} |s_{\mathrm{gt}}(\boldsymbol{v})|. \tag{23}$$

Since the vertices $\boldsymbol{v}$ lie on the reconstructed surface, $|s_{\mathrm{gt}}(\boldsymbol{v})|$ corresponds to their Euclidean distance to the ground-truth surface.

3.1.3. *Reconstruction test cases.* We first evaluate the reconstruction behavior on the flow-channel shown in Figure 8. To study the influence of the spatial resolution of the latent field and the latent dimension, we reconstruct the geometry using different tilings and compare the resulting $\mathrm{MAE}_{\mathrm{SDF}}$ values for the networks. The results are summarized in Figure 9.

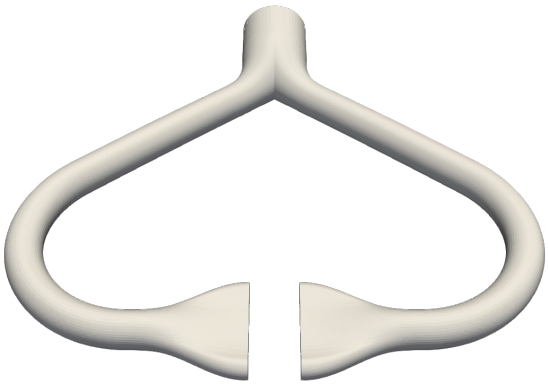


FIGURE 8. Flow-channel geometry.

For all latent dimensions, the reconstruction error decreases as the tiling is refined. This is expected, since a finer tiling increases the number of latent control points and therefore provides more local degrees of freedom. The network with $d_{\mathrm{lat}} = 8$ consistently yields the largest reconstruction error, whereas the networks with $d_{\mathrm{lat}} = 16$ and $d_{\mathrm{lat}} = 32$ perform better, while the additional improvement

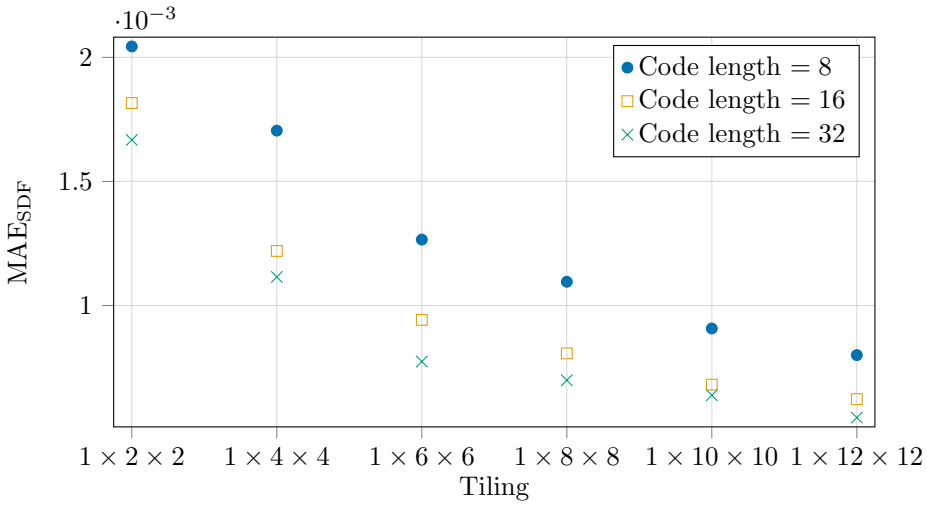
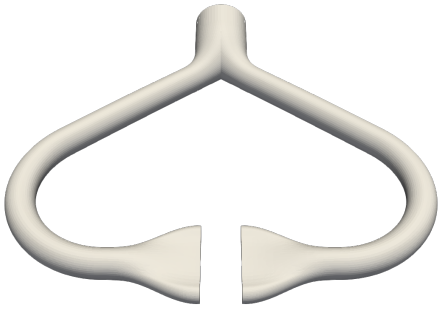


FIGURE 9. Reconstruction error of the flow-channel for different tilings and latent dimensions, measured by $\mathrm{MAE}_{\mathrm{SDF}}$.

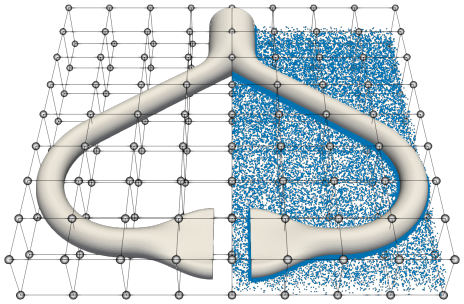
from $d_{\mathrm{lat}} = 16$ to $d_{\mathrm{lat}} = 32$ is smaller, indicating decreasing benefits from increasing the latent dimension. The best reconstruction accuracy is obtained with $d_{\mathrm{lat}} = 32$, especially for the finer tilings. This improvement comes at the cost of an increased number of latent variables. Overall, this shows that both the spatial resolution of the latent control lattice and the latent dimension are key parameters influencing the reconstruction quality.

A visual reconstruction example for the fluid-flow channel is shown in Figure 10. The reconstruction is performed using a $1 \times 8 \times 8$ tiling (voxels) and the decoder with $d_{\mathrm{lat}} = 32$. Figure 10a shows the ground-truth geometry. Figure 10b shows the geometry together with the latent control lattice and the sampled SDF points in a quarter section of the domain. Figure 10c shows the reconstructed surface, while Figure 10d visualizes the signed distance error between the reconstructed surface and the ground-truth geometry. Overall, the reconstructed geometry captures the main shape of the flow-channel well. The largest local errors occur in regions of high curvature and near sharp geometric transitions. In smoothly curved regions, the error mainly reflects the finite spatial resolution of the latent control lattice. At edges and corners, sharp features may not be reconstructed with full geometric detail during surface extraction. FlexiCubes evaluates the implicit field on a finite-resolution grid and represents the extracted zero level set by triangular faces. The finite grid resolution therefore limits how precisely sharp edges and corners can be located and represented, which can contribute to the increased local error observed in Figure 10d.

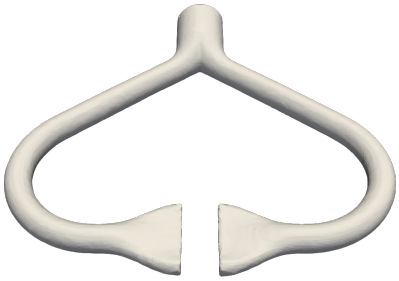
To demonstrate the ability of the network to generalize to geometrically diverse shapes, we reconstruct additional examples. These examples are shown in Figure 11. The decoder with $d_{\mathrm{lat}} = 32$ is used without retraining, only the latent control vectors are optimized for each target geometry as described in Section 2.1.4. The examples include a ship propeller [27], a car rim [28], and a dog model [26], which contain substantially different geometric features and levels of detail.



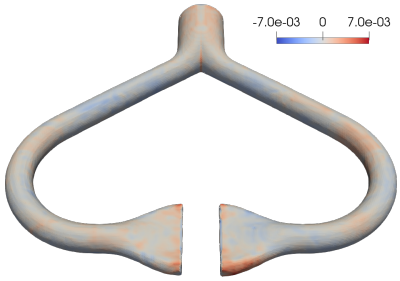
(A) Ground-truth geometry



(B) Latent control lattice and SDF sample points.



(C) Reconstructed surface.

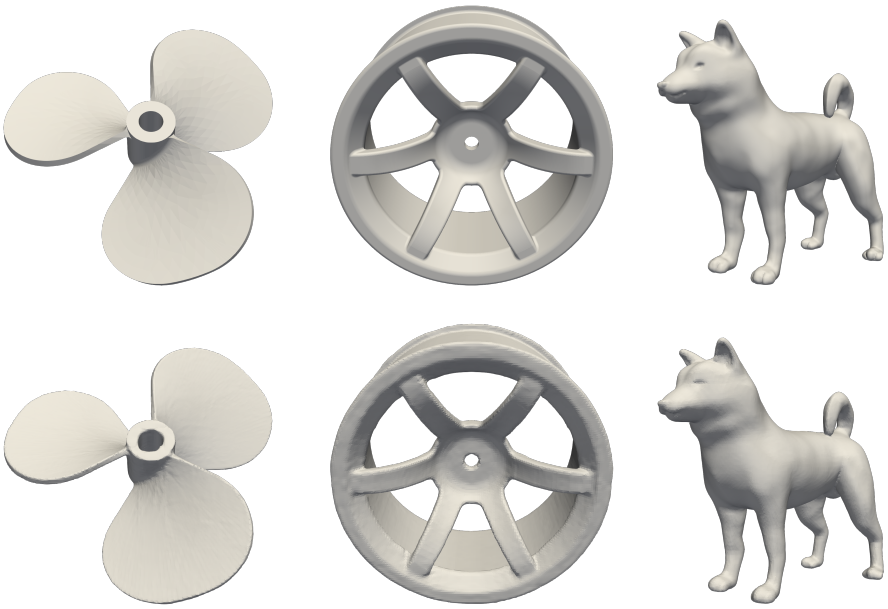


(D) Signed distance error.

FIGURE 10. Reconstruction of the fluid-flow channel using a $1 \times 8 \times 8$ tiling and a latent dimension of $d_{\text{lat}} = 32$. (a) Ground-truth geometry, (b) latent control lattice and sampled SDF points (shown in a lower-right quarter section of the domain), (c) reconstructed surface, and (d) signed distance error on the reconstructed surface.

The results show that the pretrained decoder is not restricted to specific shapes. Although the network was trained only on local shapes obtained from simple geometric primitives, it can reconstruct more complex global geometries by combining these local shapes through the continuous latent field. Detailed regions, such as the thin blades of the propeller, the spokes of the rim, or small features of the dog model, are more challenging and depend more strongly on the chosen tiling resolution. The quantitative errors for these examples are presented in Table 3. Together with the visual results, they show that the proposed representation provides sufficient geometric flexibility for a range of shapes without requiring problem-specific retraining.

Overall, the reconstruction experiments confirm that the proposed latent-field representation can accurately reconstruct both the fluid-flow channel geometry used in the optimization experiments and several geometrically distinct test cases. The results also show that the reconstruction quality improves with increasing tiling



(A) Propeller, $12 \times 12 \times 4$ tiling (B) Rim, $6 \times 12 \times 12$ tiling (C) Dog, $4 \times 12 \times 12$ tiling

FIGURE 11. Generalization of the pretrained decoder to geometrically diverse shapes. For each example, the top row shows the ground-truth geometry and the bottom row shows the reconstructed geometry. All reconstructions use the decoder with $d_{\text{lat}} = 32$.

TABLE 3. Reconstruction errors for different shapes.

Shape	d_{lat}	Tiling	Number of design variables	MAE_{SDF} ($\times 10^{-4}$)	MAE_{geom} ($\times 10^{-4}$)
Flow channel	32	$1 \times 8 \times 8$	5 184	5.73	5.31
Propeller	32	$12 \times 12 \times 4$	27 040	4.95	5.01
Rim	32	$6 \times 12 \times 12$	37 856	6.59	8.43
Dog	32	$4 \times 12 \times 12$	27 040	4.20	4.15

resolution and latent dimension. In the following optimization experiments, we further continue using the decoder with $d_{\text{lat}} = 32$.

3.2. **Optimization.** To evaluate the optimization framework introduced in Section 2.2, we consider a simple benchmark problem from fluid mechanics. More precisely, we study a three-dimensional drag minimization problem, where the objective is to reduce the drag acting on a body immersed in a viscous flow. A schematic two-dimensional sketch of the setup is shown in Figure 12.

We consider a stationary, incompressible flow in the fluid domain Ω_{F}. The flow enters the domain from the left boundary with a prescribed uniform inflow velocity $\boldsymbol{v}_\infty$ of unit magnitude and leaves the domain through the outlet boundary on the

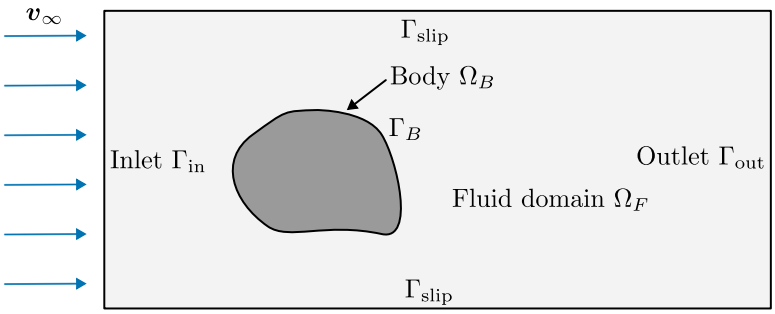


FIGURE 12. Schematic setup of the drag-minimization problem. The body is placed in a fluid domain with prescribed inflow, outflow, slip-wall, and no-slip body-boundary conditions.

right. The kinematic viscosity is set to $\nu = 1$, which, together with the chosen inflow conditions, yields a low Reynolds number regime. The top and bottom boundaries are modeled as frictionless slip walls. The immersed body occupies the domain Ω_{B} and its surface is denoted by Γ_{B}. On Γ_{B}, a no-slip boundary condition is imposed. All forward simulations and continuous-adjoint computations are performed using OpenFOAM-v2506 [19], an open-source finite-volume CFD software package. For the objective function, the enhanced surface-integral formulation [10] provides the surface shape sensitivity.

The hydrodynamic force acting on the body is computed from the Cauchy stress tensor as

$$\begin{aligned} \boldsymbol{\sigma}(\boldsymbol{u}, p) &= -p\,\boldsymbol{I} + 2\rho\,\nu\;\frac{1}{2}\left(\nabla\boldsymbol{u} + \nabla\boldsymbol{u}^{\top}\right), \\ \boldsymbol{F} &= \int_{\Gamma_{\mathrm{B}}} \boldsymbol{\sigma}(\boldsymbol{u}, p)\,\widehat{\boldsymbol{n}}\,\mathrm{d}S, \end{aligned} \tag{24}$$

where p is the pressure field, $\boldsymbol{u}$ is the fluid velocity field, ρ is the fluid density, and ν is the kinematic viscosity. The symbol $\boldsymbol{I}$ denotes the second-order identity tensor, while $\frac{1}{2}\left(\nabla\boldsymbol{u} + \nabla\boldsymbol{u}^{\top}\right)$ denotes the symmetric part of the velocity gradient. Moreover, $\widehat{\boldsymbol{n}}$ denotes the outward unit normal vector on the body surface Γ_{B}. The numerical evaluation is performed on the extracted surface mesh, for which each face has a well-defined normal. Accordingly, the discrete force integral and the surface-normal-dependent terms in the adjoint sensitivity evaluation are evaluated facewise using the corresponding face normals. This allows edges and corners to be handled numerically, although the continuum surface does not possess a unique normal at these locations.

The drag is defined as the component of the hydrodynamic force opposite to the inflow direction, i.e.,

$$D = -\frac{\boldsymbol{v}_\infty}{\|\boldsymbol{v}_\infty\|} \cdot \boldsymbol{F}. \tag{25}$$

Here, $\|\boldsymbol{v}_\infty\|$ denotes the magnitude of the inflow velocity vector.

In addition to drag minimization, two geometric constraints are imposed. First, a volume constraint is required, since otherwise the optimizer could trivially reduce the drag by continuously shrinking the body. In the present work, we therefore enforce that the body volume must not fall below its initial value, yielding

$$g_{\mathrm{V}}(\boldsymbol{d}) = V_0 - V(\boldsymbol{d}) \leq 0, \tag{26}$$

where $V(\boldsymbol{d})$ denotes the volume of the current body and V_0 is the volume of the initial design.

Second, the body has to remain at its original position in the flow domain in order to prevent rigid-body drift during the optimization. To this end, the centroid of the shape is constrained to stay within a prescribed tolerance of its initial position:

$$g_{\mathrm{c}}(\boldsymbol{d}) = \|\boldsymbol{c}(\boldsymbol{d}) - \boldsymbol{c}_0\|_2^2 - \varepsilon_{\mathrm{c}}^2 \leq 0, \tag{27}$$

where $\boldsymbol{c}(\boldsymbol{d}) \in \mathbb{R}^3$ is the centroid of the current shape, $\boldsymbol{c}_0$ is the centroid of the initial shape, and $\varepsilon_{\mathrm{c}} > 0$ is a user-defined tolerance.

In summary, the optimization problem can be written as

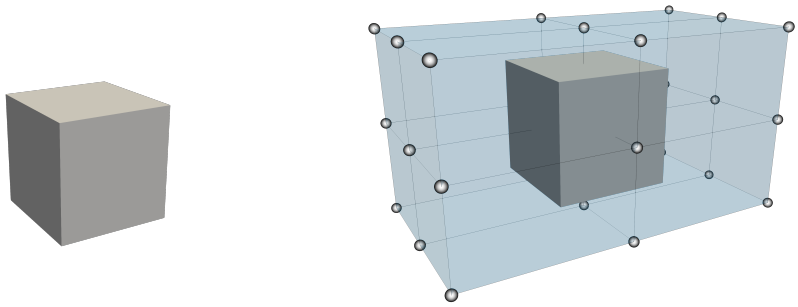
$$\begin{aligned} \min_{\boldsymbol{d}} \quad & D(\boldsymbol{d}) \\ \text{s.t.} \quad & \mathcal{R}\big(\boldsymbol{u}(\boldsymbol{d}), p(\boldsymbol{d}), \Omega_{\mathrm{F}}(\boldsymbol{d})\big) = \mathbf{0}, \\ & g_{\mathrm{V}}(\boldsymbol{d}) = V_0 - V(\boldsymbol{d}) \leq 0, \\ & g_{\mathrm{c}}(\boldsymbol{d}) = \|\boldsymbol{c}(\boldsymbol{d}) - \boldsymbol{c}_0\|_2^2 - \varepsilon_{\mathrm{c}}^2 \leq 0, \end{aligned} \tag{28}$$

where $\boldsymbol{d}$ denotes the vector of design variables and $\mathcal{R} = \mathbf{0}$ represents the stationary incompressible flow equations.

The optimization is considered converged when the relative drag change

$$\Delta_m = \frac{|D_m - D_{m-1}|}{|D_0|}, \tag{29}$$

satisfies $\Delta_m < \varepsilon_{\mathrm{D}}$ for N_{w} consecutive iterations. For the following experiments, we set $\varepsilon_{\mathrm{D}} = 2 \times 10^{-4}$ and $N_{\mathrm{w}} = 3$. The forward and adjoint CFD calculations are performed in double precision, whereas the neural decoder and differentiable surface extraction are evaluated in single precision. Comparisons using single- and double-precision evaluations of the differentiable geometry pipeline showed only minor differences in the resulting total design sensitivities. Thus, no significant influence of neural-side precision on the computed optimization gradient was observed.



(A) Initial cube geometry. (B) Design domain and latent control grid.

FIGURE 13. Setup of Experiment 1: Initial cube geometry and latent control grid used for reconstruction and optimization.

3.2.1. *Experiment 1: Cube optimization.* In the first experiment, we consider a unit cube ($\ell = 1$) as the initial geometry (Figure 13a). The cube is reconstructed using the same pretrained neural decoder as in the reconstruction experiments; no retraining is performed for the optimization problem. We define a design domain

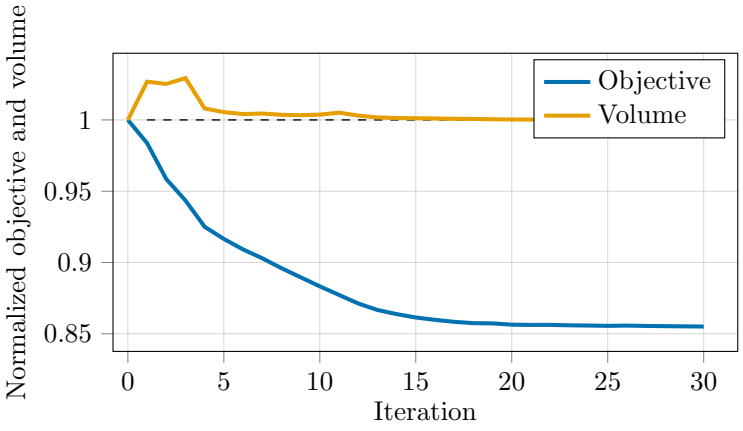
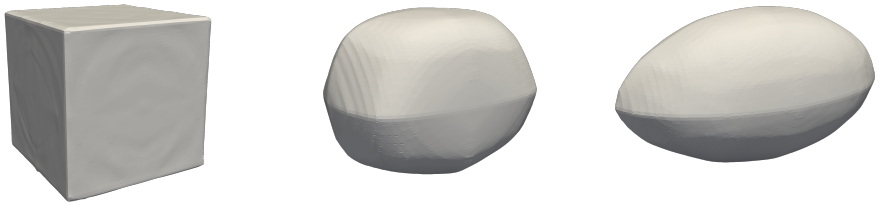


FIGURE 14. Optimization history for Experiment 1: Normalized drag objective and normalized body volume over the optimization iterations.

with dimensions $3 \times 1.5 \times 1.5$, which provides sufficient space for the optimization. The domain is split into $2 \times 2 \times 2$ voxels, resulting in a latent control grid with $3 \times 3 \times 3$ latent control points. Since each node is assigned a latent vector of dimension $d_{\text{lat}} = 32$, this leads to a total of 864 design variables. Figure 13b shows the design domain together with the latent control grid. Figure 14 shows the normalized evolution of the drag and the volume of the geometry. The drag decreases steadily over the course of the optimization until the convergence criterion in Equation (29) is met after 30 iterations, while the normalized volume remains close to its initial value and satisfies the prescribed volume constraint. Overall, the drag is reduced by approximately 15% compared to the initial design. In addition, the centroid constraint remained satisfied throughout the optimization. The smooth decrease of the objective value indicates that the proposed latent-space formulation is capable of identifying meaningful shape modifications within a sufficiently smooth latent space.

Figure 15 shows the geometric evolution at iteration 0, iteration 10, and the final iteration. Starting from the initial cube, the optimizer gradually smooths the sharp edges and corners and transforms the body into a more rounded and streamlined shape. The intermediate design already shows a substantial reduction of the cubic features, while the final design approaches an ellipsoidal geometry. The evolution toward an elongated, rounded body is qualitatively consistent with classical minimum-drag results for Stokes flow, which predict a body similar to a prolate spheroid, although with pointed ends [22]. Overall, the experiment confirms that the proposed neural implicit representation is sufficiently smooth, robust, and expressive for adjoint-based fluid-flow shape optimization.

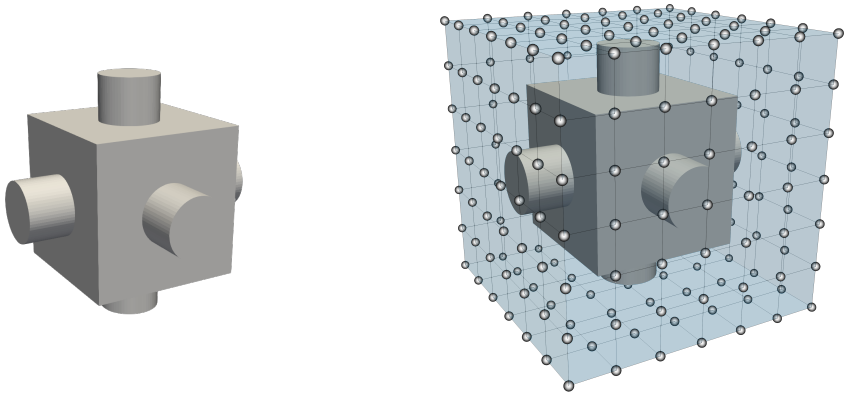
3.2.2. *Experiment 2: Comparison with free-form deformation.* To further assess the proposed parameterization, we compare it with a classical FFD approach. In FFD, the geometry is embedded into a volumetric spline lattice, and the positions of the lattice control points are used as design variables. This yields a smooth and



(A) Initial geometry. (B) Iteration 10. (C) Final geometry.

FIGURE 15. Geometric evolution of the cube in Experiment 1 during drag minimization.

well-established parameterization for shape optimization, but the admissible deformations remain linked to the chosen control lattice. The aim of this experiment is therefore to compare the optimization performance and geometric flexibility of the classical FFD formulation with those of the proposed neural SDF-based method on the same drag minimization problem. As test case, we consider a slightly more



(A) Initial geometry. (B) FFD control lattice with $7\times7\times7$ control points.

FIGURE 16. Setup of Experiment 2: Initial geometry and FFD control lattice used for the comparison.

complex extension of the cube from Experiment 1 (Section 3.2.1), namely a cube with cylindrical protrusions on all six faces. The initial geometry is shown in Figure 16a. For the FFD parameterization, we use a quadratic volumetric spline with either $5 \times 5 \times 5$ or $7 \times 7 \times 7$ control points. This simple rectangular FFD lattice is chosen intentionally, because it is a low-manual-effort setup and therefore provides a fair comparison to the proposed method, which also avoids problem-specific construction of design variables. The finer $7 \times 7 \times 7$ control lattice is illustrated in Figure 16b. Apart from the geometry parameterization, the optimization workflow is identical to that of the proposed approach. In the FFD case, we impose an additional inequality constraint on the Jacobian determinant of the deformation in order to prevent self-penetration and preserve the validity of the parameterization. Table 4 summarizes the comparison in terms of the number of design variables, the

TABLE 4. Comparison of the classical FFD parameterization and the proposed neural SDF-based parameterization for the drag minimization problem.

Method	Design variables	Iterations	Drag reduction [%]
FFD ($5 \times 5 \times 5$ control points)	375	46	14.2
FFD ($7 \times 7 \times 7$ control points)	1029	32	15.7
Proposed neural SDF method	864	33	16.7

iterations required for convergence, and the achieved drag reduction. The coarser FFD parameterization with $5 \times 5 \times 5$ control points uses fewer design variables but also yields the smallest drag reduction. Increasing the FFD resolution to $7 \times 7 \times 7$ improves the optimization result, but also increases the number of design variables to 1029. The proposed neural SDF method achieves the largest drag reduction of 16.7% while using only 864 design variables and converging after 33 iterations. Hence, the neural SDF formulation outperforms even the finer FFD discretization while requiring fewer design variables. To assess the numerical uncertainty of the reported drag values and verify that the comparison between the two parameterizations is not affected by the mesh resolution, a three-level mesh-refinement study was performed. The corresponding drag reductions are shown in Figure 17.

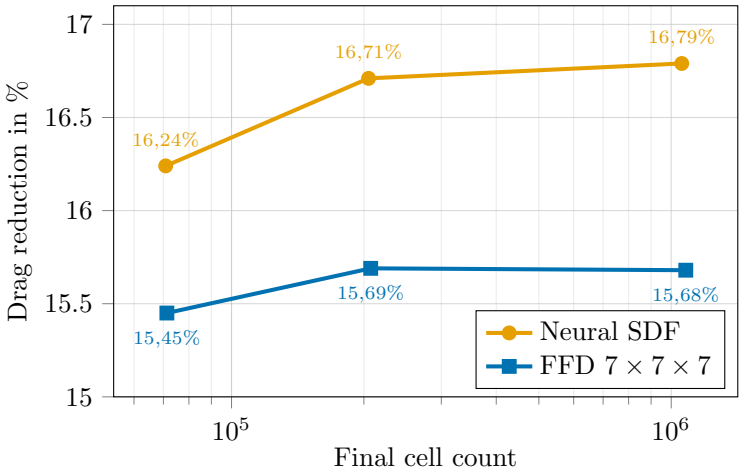
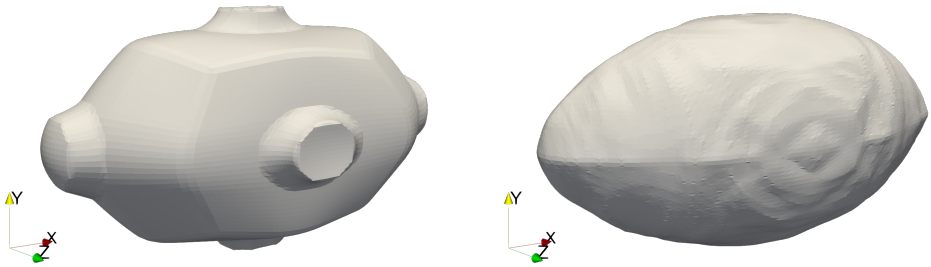


FIGURE 17. Drag reduction of the optimized neural-SDF and FFD designs at the three mesh-refinement levels.

The study uses coarse, intermediate, and fine meshes with approximately 7×10^4, 2×10^5, and 1.1×10^6 cells, respectively. The intermediate mesh corresponds to the resolution used for the optimization results. As shown in Figure 17, the neural-SDF design retains a lower drag than the FFD design at all three mesh resolutions. Refinement from the intermediate to the fine mesh changes the final drag by less than 0.1% for both geometries, indicating that the observed ranking is robust with

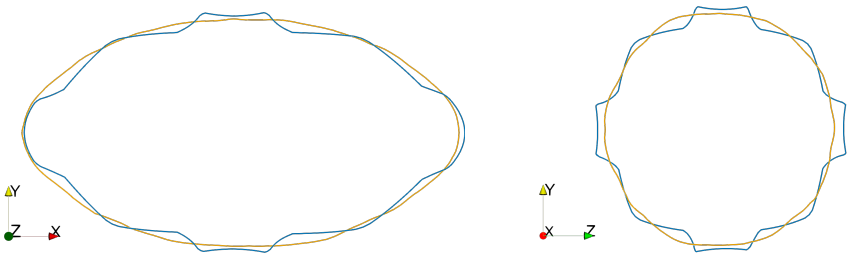
respect to mesh refinement. All primal solutions in the study satisfied the residual-control criterion of 10^{-5} within 606–924 SIMPLE iterations.



(A) Final geometry obtained with FFD ($7\times 7\times 7$ control points).

(B) Final geometry obtained with the proposed neural SDF method.

FIGURE 18. Comparison of the final geometries obtained with classical FFD and the proposed neural SDF-based parameterization.



(A) Cross-section view 1.

(B) Cross-section view 2.

FIGURE 19. Overlay of two orthogonal cross-sections through the centroid of the optimized geometries. The FFD result is shown in blue, while the result obtained with the proposed neural SDF method is shown in orange.

A visual comparison of the optimized geometries is given in Figure 18. The FFD result is smoother than the initial shape, but still retains more pronounced remnants of the original protrusions and appears more restricted by the underlying spline lattice. In contrast, the geometry obtained with the proposed method evolves into a smoother and more streamlined body. This is further illustrated in Figure 19, where two orthogonal cross-sections through the centroid of the optimized geometries are overlaid. The two solutions are comparable, but the neural SDF result exhibits smoother contours and greater local geometric freedom. Overall, this experiment indicates that the proposed latent-space parameterization provides a more expressive design space than classical FFD and can achieve better aerodynamic performance without requiring a larger number of design variables.

3.2.3. *Experiment 3: Topological changes.* In the third experiment, we investigate if the proposed neural SDF-based parameterization is able to realize topological changes during the optimization. For this purpose, we consider a modified version

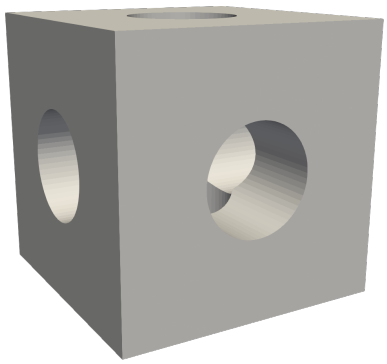
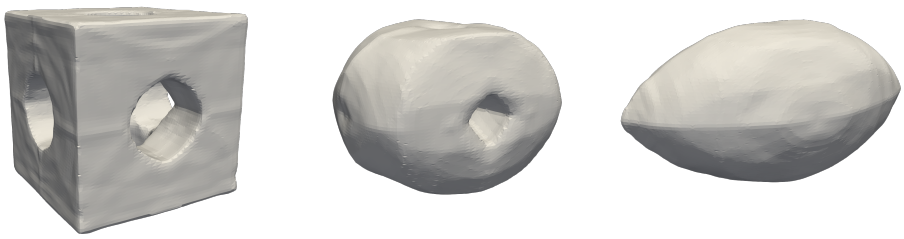


FIGURE 20. Initial geometry for Experiment 3. The body is based on the cube from Experiment 1, but cylindrical through-holes are introduced in all three coordinate directions.

of the cube geometry used in Experiment 1 (3.2.1). The initial body is again based on a unit cube, but cylindrical through-holes are introduced in all three coordinate directions, such that openings are present on each face. The resulting initial geometry is shown in Figure 20. This setup is chosen because closing or removing the holes cannot be performed without a change in topology. The optimization problem, boundary conditions, and constraints are identical to those used in the previous drag minimization experiments. The body volume is constrained to remain at least as large as the initial volume, and the centroid constraint prevents rigid-body drift of the geometry inside the flow domain. The optimization converges after 21 iterations and achieves a drag reduction of 25.4% compared to the initial design. The volume constraint remains satisfied throughout the optimization, while the centroid displacement stays within the prescribed tolerance. Figure 21 shows



(A) Initial geometry (B) Iteration 3 (C) Final geometry

FIGURE 21. Geometric evolution of the perforated cube during drag minimization. During the optimization, the holes are progressively reduced and finally closed, while the body evolves into a smoother and more streamlined shape.

the geometric evolution during the optimization. Starting from the perforated cube, the optimizer first smooths the sharp cube edges and reduces the size of the openings. Already after three iterations, the initially cylindrical holes are significantly

deformed and partially closed. In the final optimized geometry, the holes have disappeared completely and the body has evolved into a closed, rounded, and more streamlined shape. This behavior demonstrates that the proposed parameterization can also modify the topology of the represented geometry through modifications of the underlying implicit field.

For the three experiments, the one-time latent reconstruction required 173–285 s. All timings were measured on a single machine equipped with an Intel Xeon Gold 6326 CPU (2.90 GHz, 16 cores) and an NVIDIA GeForce RTX 4090 (24 GB). The forward and adjoint solutions were computed using 16 MPI ranks. The initialization corresponds to approximately 3.8–4.5% of the complete optimization-loop time.

4. **Conclusion.** In this work, we presented a neural implicit geometry representation framework for gradient-based shape optimization. The proposed approach represents geometries by a signed distance function approximated through a neural decoder and controlled by a continuous latent field. Since the decoder is trained only once on simple geometric primitives, the method does not require retraining for new target geometries. During shape optimization, the latent control variables define the design space, avoiding the manual construction of problem-specific design variables. The continuous local latent field provides flexibility and local geometric control within this design space.

The reconstruction experiments demonstrated the ability of the network to generalize to geometrically diverse shapes. These results indicate that the decoder learns reusable local shape patches, which can be combined through the continuous latent field to describe complex global geometries. The reconstruction accuracy improved with increasing tiling resolution and latent dimension, highlighting the importance of both spatial and latent resolution for the expressiveness of the representation.

The optimization experiments further showed that the proposed representation can be successfully integrated into a gradient-based shape optimization workflow. The method was tested on three drag-minimization problems with different initial geometries. Starting from a simple cube, the method produced a smooth and physically meaningful shape evolution. The drag was reduced while the imposed volume and centroid constraints remained satisfied. In the second test case, the method was compared with classical free-form deformation. The proposed neural SDF-based parameterization achieved a larger drag reduction while using fewer design variables. The optimized geometry also showed smoother contours and greater local geometric flexibility. The perforated-cube example further showed that the implicit representation can realize topological changes during optimization. For the investigated cases, the initial latent reconstruction represented only a comparatively small overhead relative to the iterative online optimization. The current implementation is research-oriented and most likely has potential for optimized computational performance.

Overall, the results suggest that neural implicit geometry representations provide a promising alternative to classical geometry parameterizations in shape optimization. The combination of local control, differentiability, and reusable training on simple geometric primitives offers a flexible design space and a largely automated workflow. Some aspects remain to be investigated in more detail, including the influence of the decoder architecture and the training-data distribution. Furthermore, the numerical examples focused on simple benchmark problems. Therefore,

future work should investigate the application of the framework to more complex engineering problems and industrial geometries. Despite these limitations, the proposed framework highlights the potential of neural implicit geometry representations to make gradient-based shape optimization more flexible and less dependent on manually constructed design spaces.

Acknowledgments. This research was funded in part by the Austrian Science Fund (FWF) 10.55776/I6957, a project that is embedded into the TRR 402/1-525069572 funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation).

Code and data availability. The source code used for the numerical experiments is publicly available in the `DeepShapeOpt` repository and is archived on Zenodo [6]. The archived `DeepSDFStruct` fork used as a software dependency is available on Zenodo [12]. The training data for the neural decoder and the result data supporting the numerical experiments are available from the TU Wien Research Data repository [7].

REFERENCES

[1] L. Abergo, M. Morelli and A. Guardone, Aerodynamic shape optimization based on discrete adjoint and RBF, *Journal of Computational Physics*, **477** (2023), 111951. https://www.sciencedirect.com/science/article/pii/S0021999123000463.

[2] I. Antonau, S. Warnakulasuriya, K.-U. Bletzinger, F. M. Bluhm, M. Hojjat and R. Wüchner, Latest developments in node-based shape optimization using vertex morphing parameterization, *Structural and Multidisciplinary Optimization*, **65** (2022), 198.

[3] H. R. Attar, A. Foster and N. Li, Implicit neural representations of sheet stamping geometries with small-scale features, *Engineering Applications of Artificial Intelligence*, **123** (2023), 106482. https://www.sciencedirect.com/science/article/pii/S0952197623006668.

[4] R. Chabra, J. E. Lenssen, E. Ilg, T. Schmidt, J. Straub, S. Lovegrove and R. Newcombe, Deep local shapes: Learning local SDF priors for detailed 3D reconstruction, in *Computer Vision – ECCV 2020 (eds. A. Vedaldi, H. Bischof, T. Brox and J.-M. Frahm)*, Springer International Publishing, Cham, (2020), 608-625.

[5] S. D. Daxini and J. M. Prajapati, Parametric shape optimization techniques based on meshless methods: A review, *Structural and Multidisciplinary Optimization*, **56** (2017), 1197-1214.

[6] L. Freinberger, DeepShapeOpt: Shape optimization with neural implicit geometry representation, Version 0.1.0, archived software release, *Zenodo*, 2026.

[7] L. Freinberger, Training data for the neural decoder and result data for neural implicit shape optimization, *TU Wien Research Data*, 2026. https://doi.org/10.48436/12y18-j6236.

[8] H. Gottschalk and M. Reese, An analytical study in multi-physics and multi-criteria shape optimization, *Journal of Optimization Theory and Applications*, **189** (2021), 486-512.

[9] M. Hojjat, E. Stavropoulou and K.-U. Bletzinger, The Vertex Morphing method for node-based shape optimization, *Computer Methods in Applied Mechanics and Engineering*, **268** (2014), 494-513. https://www.sciencedirect.com/science/article/pii/S0045782513002673.

[10] I. S. Kavvadias, E. M. Papoutsis-Kiachagias and K. C. Giannakoglou, On the proper treatment of grid sensitivities in continuous adjoint methods for shape optimization, *Journal of Computational Physics*, **301** (2015), 1-18. https://www.sciencedirect.com/science/article/pii/S0021999115005318.

[11] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, 2017. https://arxiv.org/abs/1412.6980.

[12] M. Kofler and L. Freinberger, DeepSDFStruct: Differentiable signed-distance-function and spline-lattice tooling, Version 0.1.0, archived software release, *Zenodo*, 2026.

[13] M. Kofler, M. Giritsch and S. Elgeti, Structural optimization of lattice structures using deep neural networks as geometry representation, *Graphical Models*, **142** (2025), 101307. https://www.sciencedirect.com/science/article/pii/S1524070325000542.

[14] Y. Kwon and N. Kang, Three-dimensional deep shape optimization with a limited dataset, *Engineering Applications of Artificial Intelligence*, **159** (2025), 111504. https://www.sciencedirect.com/science/article/pii/S0952197625015064.
[15] H. J. Lamousin and N. N. Waggenspack, NURBS-based free-form deformations, *IEEE Computer Graphics and Applications*, **14** (1994), 59-65.
[16] J. Lee, S. Hube and S. Elgeti, Neural networks vs. splines: Advances in numerical extruder design, *Engineering with Computers*, **40** (2024), 989-1004.
[17] B. Mohammadi and O. Pironneau, *Applied Shape Optimization for Fluids*, 2nd edition, Numerical Mathematics and Scientific Computation, Oxford University Press, Oxford, 2010.
[18] P. M. Müller, N. Kühl, M. Siebenborn, K. Deckelnick, M. Hinze and T. Rung, A novel p-harmonic descent approach applied to fluid dynamic shape optimization, *Structural and Multidisciplinary Optimization*, **64** (2021), 3489-3503.
[19] OpenFOAM, 2025. https://www.openfoam.com/.
[20] C. Othmer, A continuous adjoint formulation for the computation of topological and surface sensitivities of ducted flows, *International Journal for Numerical Methods in Fluids*, **58** (2008), 861-877.
[21] J. J. Park, P. Florence, J. Straub, R. Newcombe and S. Lovegrove, DeepSDF: Learning continuous signed distance functions for shape representation, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA*, (2019), 165-174.
[22] O. Pironneau, On optimum profiles in Stokes flow, *Journal of Fluid Mechanics*, **59** (1973), 117-128.
[23] T. W. Sederberg and S. R. Parry, Free-form deformation of solid geometric models, in *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '86, Association for Computing Machinery, New York, NY, USA, (1986), 151-160,
[24] T. Shen, J. Munkberg, J. Hasselgren, K. Yin, Z. Wang, W. Chen, Z. Gojcic, S. Fidler, N. Sharp and J. Gao, Flexible isosurface extraction for gradient-based mesh optimization, *ACM Transactions on Graphics (TOG)*, **42** (2023), Article No. 37, 1-16.
[25] K. Svanberg, The method of moving asymptotes—a new method for structural optimization, *International Journal for Numerical Methods in Engineering*, **24** (1987), 359-373.
[26] Thingiverse.com, My little shiba, by layerone. https://www.thingiverse.com/thing:1308678.
[27] Thingiverse.com, Parametric Ship Propeller/ Rc propeller/ Marine Propeller, by Fysik_klubben. https://www.thingiverse.com/thing:6906448.
[28] Thingiverse.com, RC Rim, by Attila_D. https://www.thingiverse.com/thing:1328760.
[29] B. D. Upadhyay, S. S. Sonigra and S. D. Daxini, Numerical analysis perspective in structural shape optimization: A review post 2000, *Advances in Engineering Software*, **155** (2021), 102992. https://www.sciencedirect.com/science/article/pii/S0965997821000211.
[30] F. Wein, P. D. Dunning and J. A. Norato, A review on feature-mapping methods for structural optimization, *Structural and Multidisciplinary Optimization*, **62** (2020), 1597-1638.
[31] S. Xu, W. Jahn and J.-D. Müller, CAD-based shape optimisation with CFD using a discrete adjoint, *International Journal for Numerical Methods in Fluids*, **74** (2014), 153-168.
[32] Z. Zhang, W. Yao, Y. Li, W. Zhou and X. Chen, Topology optimization via implicit neural representations, *Computer Methods in Applied Mechanics and Engineering*, **411** (2023), 116052. https://www.sciencedirect.com/science/article/pii/S0045782523001767.

Received July 7, 2026; revised August 24, 2026.